

Chapter 16. Business Intelligence with Power BI

Model the data once, write measures that hold under any filter, design a page that tells the truth, and let it refresh itself.

Approved chapter v1

Contents

Chapter 16. Business Intelligence with Power BI

Why this matters

In plain English

16.1 What Power BI is, and what it costs

16.2 Getting Riverstone's data in

16.3 The model: a star schema and a date table

16.4 DAX from zero

16.5 `CALCULATE` and filter context

16.6 Time intelligence

16.7 Designing the report page

16.8 A worked example: Riverstone's monthly pack

16.9 Publishing, refresh, and sharing

16.10 Row-level security

16.11 Performance and the mistakes that make numbers wrong

16.12 The same ideas in other tools

16.13 Where BI tools are heading

Common mistakes and how to spot them

In the real world: the ₹2.3 crore that vanished

Tools

The project: replace Riverstone's monthly pack with a live report

Timed challenge: the Q4 branch page (45 minutes)

You've got it when...

Recap

Practice exercises

Key terms

Where this leads

Answers to practice exercises

Chapter 16. Business Intelligence with Power BI

Part II – The Analyst

Chapter at a glance

You will learn to: explain what a BI tool does that a spreadsheet can't, and which Power BI licence buys what · load Riverstone's full dataset with Power Query and choose between Import and DirectQuery · build a star schema with a proper date table · write DAX measures from zero, and know when a measure beats a calculated column · use `CALCULATE` to change filter context deliberately · build time intelligence (year to date, same period last year, year-over-year growth) that survives slicers · design a report page with Chapter 15's principles, using cards, slicers, tooltips, drill-through, and bookmarks · publish, schedule refresh, use a gateway, and share with an app · apply row-level security so each region sees only its own customers · keep a model fast and avoid the modelling mistakes that make numbers wrong · recognize the same ideas in Tableau and Looker Studio.

Before you start: Chapter 11 (Power Query, the Data Model, and DAX in Excel), Chapter 13 (SQL aggregation and the `sales_lines` view), Chapter 14 (cleaning: this chapter assumes clean data), and Chapter 15 (chart choice, color, titles: this chapter applies them rather than repeating them). The Python block (Chapters 17 and 18) comes before this chapter in the reading order, so pandas is assumed knowledge where it appears.

Time needed: 18–22 hours, spread over two to three weeks.

Tools: Power BI Desktop, free, **Windows only**. On macOS or Linux you need a Windows virtual machine, a cloud Windows desktop, or a colleague's machine; the browser version of Power BI can view and lightly edit reports but cannot build the model this chapter teaches, and it needs a work or school account. PostgreSQL or MySQL with `riverstone_full` (or the CSV files) for the data.

Practice data: the full Riverstone dataset (`companion/full/`), plus `companion/ch16/` which holds the chapter's DAX measures as text files, the expected value of every measure, the SQL that produces those values, and a `UserRegion` table for the security section.

Why this matters

By the end of Chapter 15 you can clean a dataset, analyze it, and draw an honest chart. There's one thing left that separates an analyst's work from a report people rely on: **it has to stay true next month without you.**

That's what a BI tool does. You build the model once, and the report refreshes itself: new orders arrive, the numbers move, the branch heads open the same link and see today's figures. Nobody emails a workbook. Nobody asks "is this the latest version?" The 25 hours a year you'd spend rebuilding a monthly pack become 25 hours of analysis.

Power BI is the most common BI tool in Indian companies, and the one most analyst job descriptions name. The skills transfer: Tableau, Looker Studio, Qlik, and Metabase differ in syntax and menus, but every one of them asks you to model data, write measures, and design a report. Learning Power BI properly means learning that shape of work.

This chapter builds one thing: a live Riverstone sales report on the full three-year dataset, with measures that match to the rupee, security so each region sees only its own customers, and a refresh schedule. Then it shows what goes wrong, which is mostly modelling, not charting.

About screenshots. Power BI's interface changes every month. This chapter describes what to click in words and explains the ideas behind it, so it stays useful when the buttons move. Where a menu name might have changed, check Microsoft's current documentation.

In plain English

A spreadsheet is a **notebook**: you write the numbers in, and they stay as you left them. A BI tool is a **kitchen**: you write the recipe, and the dish gets made again with whatever arrives that morning.

Building a Power BI report is setting up that kitchen:

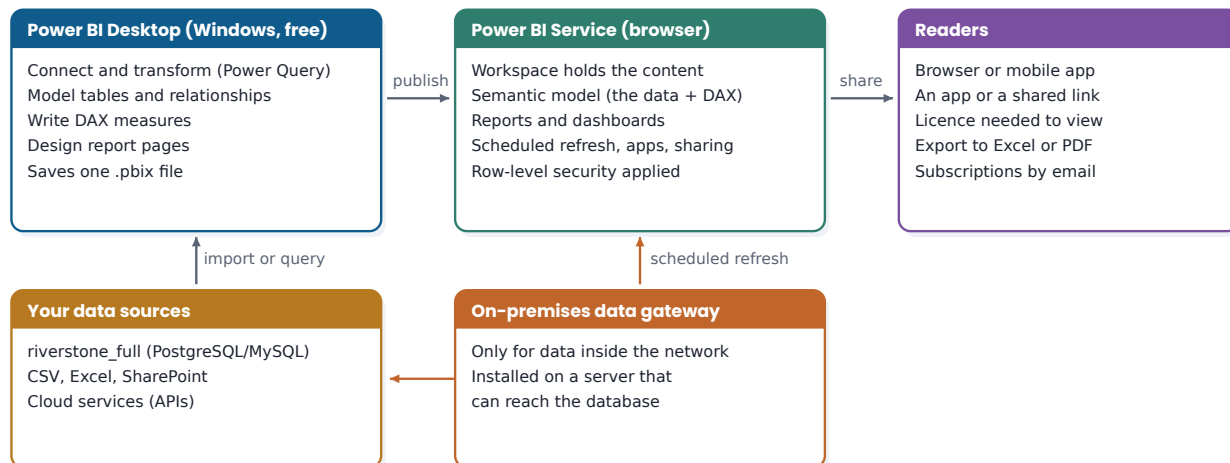
- **Power Query** is the prep station. Wash, chop, standardize: the cleaning steps from Chapter 14, recorded once so they repeat with every delivery.
- **The model** is how ingredients are organized: one shelf for customers, one for products, one for dates, and the big crate of order lines in the middle. Put things on the right shelves and every recipe is quick; pile them up anywhere and every dish takes an hour.
- **DAX measures** are the recipes: "net revenue" is written once and cooked correctly whether you ask for one region, one month, or the whole company.
- **The report page** is the plate: what the guest sees, arranged for them, not for you.
- **Refresh** is the standing delivery order, and **row-level security** is the rule that each table gets served only its own dish.

The most common beginner mistake is to keep working like a notebook: pasting values into the model, writing a formula for each cell, and making a new page for each region. The kitchen only pays off when the recipe is written once and the ingredients keep arriving.

16.1 What Power BI is, and what it costs

The pieces

The pieces of Power BI, and where each one runs



Desktop builds it; the Service runs it on a schedule and shares it. Nothing you build is visible to anyone until you publish.

Figure 16.1 – Desktop builds it, the Service runs it. Nothing you build is visible to anyone until you publish.

- **Power BI Desktop** is a free Windows application. It contains Power Query (the same engine as Excel's, Chapter 11), the data model, the DAX formula language, and the report designer. Its file is a .pbix .
- **The Power BI Service** (`app.powerbi.com`) is the cloud side: **workspaces** hold your published content, a **semantic model** (formerly "dataset") holds the data and the DAX, **reports** hold the pages, **dashboards** pin selected visuals from several reports, and an **app** is the polished, read-only package you hand to the business.
- **The on-premises data gateway** is a small program installed on a machine inside your network so the Service can refresh from a database it can't otherwise reach.
- **Power BI is part of Microsoft Fabric**, Microsoft's wider analytics platform. For an analyst, that mostly changes the words on the pricing page and the menus around the workspace; the Desktop, model, and DAX work in this chapter is unchanged.

Licences, as of writing

Licensing decides who can see what you build, so check the current rules before promising anything. At the time of writing (September 2026), Microsoft's published list prices are:

Licence	Price (US list)	What it allows
Free	₹0	Power BI Desktop, and personal workspaces in the Service. No sharing.
Pro	\$14 per user per month	Publishing to shared workspaces, sharing with other Pro users, apps, 8 scheduled refreshes a day. Included with Microsoft 365 E5.

Licence	Price (US list)	What it allows
Premium Per User (PPU)	\$24 per user per month	Pro plus larger models, 48 refreshes a day, paginated reports, deployment pipelines. Everyone who views PPU content needs PPU.
Fabric capacity (F SKUs)	From about \$263 a month (F2), pay as you go	Shared compute instead of per-user licences. From F64 upward, viewers with free licences can read content.

Both per-user prices rose on 1 April 2025 (Pro from \$10 to \$14, PPU from \$20 to \$24) and were unchanged through 2026. The older Premium per-capacity P SKUs are no longer sold to new customers; Microsoft directs enterprise buyers to Fabric capacity instead. Prices are US list prices before tax and vary by country and agreement.

Two practical consequences:

1. **You can learn the whole of this chapter for free.** Desktop costs nothing, and everything up to publishing works without a licence. Only sharing needs Pro.
2. **Count the readers, not the authors.** A report for 300 branch staff at ₹1,200 a month each is a real budget line; the F64 threshold is where organizations switch to capacity so that viewers don't need paid seats.

When a BI tool is the right answer

Situation	Better answer
The same report, refreshed regularly, read by several people	Power BI
Many people need to slice the same data themselves	Power BI
Numbers must be consistent across teams (one definition of revenue)	Power BI (a shared semantic model)
A one-off analysis nobody will repeat	A spreadsheet or a notebook
Heavy statistics, modelling, or machine learning	Python (Part IV)
Ad-hoc data exploration by an analyst	SQL
A document with commentary and exhibits	A written report (Chapter 20)

Watch out: Power BI is not a database. It reads from sources and keeps a copy for speed. It's not where data should be corrected, and a model is not a substitute for a warehouse (Part III). If the fix belongs upstream, fix it upstream (Chapter 14, section 14.10).

16.2 Getting Riverstone's data in

Connect

Home → Get data lists the connectors. For this chapter:

- **PostgreSQL database:** server `localhost` , database `riverstone_full` . Power BI needs the Npgsql provider on older Desktop versions; recent versions ship it.
- **MySQL database:** needs the MySQL Connector/NET package installed, which Desktop will prompt for.
- **Text/CSV or Folder:** the files in `companion/full/` work equally well if you'd rather not install a database.

Pick the tables `orders` , `order_items` , `customers` , `products` , `employees` , and `sales_targets` .

Transform Data opens Power Query; **Load** skips it, which you rarely want.

Import or DirectQuery

Power BI asks how to connect:

Mode	How it works	Use it when
Import	Data is copied into the model's in-memory engine, compressed. Fastest reports; needs refresh to update.	Almost always. This chapter uses Import.
DirectQuery	Every visual sends a query to the source. Always current; as fast as your database, and many DAX features are limited.	Data too large to import, or a real-time requirement
Composite / Dual	Some tables imported, some DirectQuery	Large fact table with small dimensions
Live connection	To an existing published semantic model or Analysis Services	Reusing a model someone else owns

Riverstone's 209,006 order lines are tiny for Import: the model compresses to a few megabytes and every visual responds instantly. Import until you have a reason not to.

Power Query here versus Chapter 14

The Power Query in Desktop is the one you learned in Chapter 11 and used for cleaning in Chapter 14, with the same ribbon and M language. The same rules apply: profile the whole dataset (**View → Column quality, Column distribution, Column profile**, and switch profiling to the entire dataset), and transform in recorded steps.

What changes is **where cleaning belongs**. Prefer, in order:

1. **Fix it in the source system**, so everyone benefits (Chapter 14's cleaning log exists for exactly this).
2. **Fix it in a database view or a warehouse table**, so every tool sees the same clean data.
3. **Fix it in Power Query**, when you can't do either.
4. **Fix it in DAX**, only for things that must respond to filters.

For Riverstone, the sensible split is: use the `sales_lines` view idea from Chapter 13 as the fact table (non-cancelled lines with `net_revenue` and `product_cost` computed), and add in Power Query only what the report needs: a `region` column on customers, and the branch mapping from Chapter 14.

Query folding matters here. When Power Query can translate your steps into SQL, the database does the work and only the result travels. Right-click a step → **View Native Query**: if it's greyed out, folding stopped at that step. Filters, column removal, renames, and simple type changes usually fold; custom

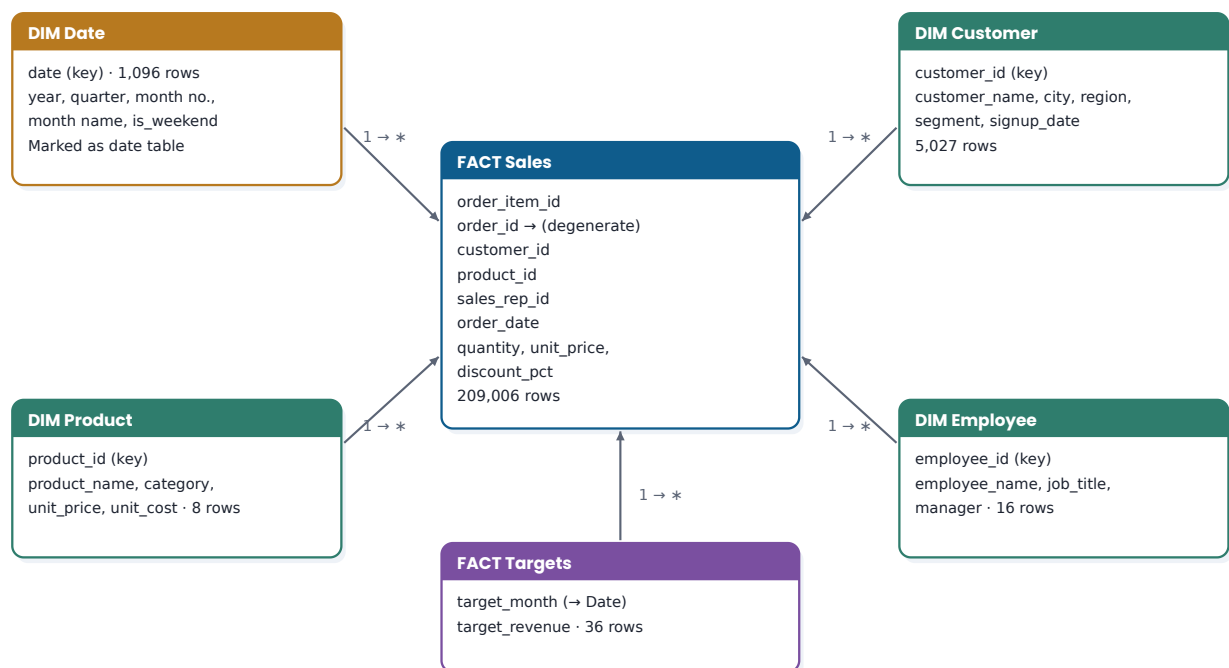
M functions and index columns usually break it. Put the folding steps first.

Tool note: don't import what you don't need. Remove columns you'll never show, filter to the years you report on, and drop helper columns before loading. Every column loaded costs memory and refresh time, and a column with many distinct values (an ID, a timestamp to the second) costs the most.

16.3 The model: a star schema and a date table

A **model** is a set of tables joined by relationships. Its shape decides whether DAX is simple or painful.

Riverstone's star schema in Power BI



One row per order line in the fact table; one row per thing in each dimension. Filters flow from the dimensions down into the facts, which is why every table joins to Date rather than to each other. Both fact tables use the Date dimension, at different grains.

Figure 16.2 — A star schema: one fact table per business event, one dimension per thing you filter or group by, everything joined through keys, and both fact tables sharing the Date dimension.

Facts and dimensions

- A **fact table** holds events, one row per event, with keys and numbers: here, one row per order line, with `customer_id`, `product_id`, `sales_rep_id`, `order_date`, quantity, price, discount. It's long and narrow, and it's where the measures aggregate.
- A **dimension table** holds the things you slice by, one row each, with descriptive columns: Customer, Product, Employee, Date. Short and wide.
- **Relationships** run one-to-many from the dimension to the fact, and filters flow **down** that arrow: filtering Customer filters Sales, not the other way around (unless you switch cross-filtering to both, which you should avoid until you know why you need it).

This shape is called a **star schema** because the fact table sits in the middle with dimensions around it. It's not a Power BI quirk: it's the standard analytical model (Chapter 28 covers the theory, and Chapter 11's Data Model section showed the Excel version).

Riverstone's model from the database:

Table	Role	Rows	Key
Sales (order lines joined to orders)	Fact	209,006	—
Targets (sales_targets)	Fact	36	target_month
Date	Dimension	1,096	date
Customer	Dimension	5,027	customer_id
Product	Dimension	8	product_id
Employee	Dimension	16	employee_id

Two fact tables at different grains (order lines by day, targets by month) is normal. They connect through the shared Date dimension, which is why a date table isn't optional.

The date table

Create it in Power Query (a **Blank Query** with `List.Dates`) or in DAX:

```
Date =
ADDCOLUMNS (
    CALENDAR ( DATE ( 2023, 1, 1 ), DATE ( 2025, 12, 31 ) ),
    "Year", YEAR ( [Date] ),
    "Quarter", "Q" & FORMAT ( [Date], "Q" ),
    "Month No", MONTH ( [Date] ),
    "Month", FORMAT ( [Date], "MMM" ),
    "Year Month", FORMAT ( [Date], "YYYY-MM" ),
    "Financial Year", "FY" & IF ( MONTH ( [Date] ) >= 4, YEAR ( [Date] ) + 1, YEAR ( [Date] ) )
)
```

That's **1,096 rows** for 2023–2025. Then:

1. **Mark it as a date table:** select the table → **Table tools** → **Mark as date table**, pointing at `Date[Date]` . Time-intelligence functions need this.
2. **Join it** to `Sales[order_date]` and to `Targets[target_month]` .
3. **Sort month names properly:** select the `Month` column → **Column tools** → **Sort by column** → **Month No**, or every chart will run April, August, December.
4. **Include whole years.** A date table that stops mid-December breaks year-to-date calculations.
5. **Add a financial-year column** if the business reports April–March, as most Indian companies do. Riverstone's targets are calendar months, so this chapter uses calendar years, but the column is there for exercises.

Hide, rename, and format

The model is the interface everyone else uses, so treat it like one:

- **Hide** raw key columns (`customer_id` , `product_id` , `order_id`) from report view: right-click → **Hide in report view**. They're for relationships, not for dragging onto a page.
- **Rename** columns into business language: `customer_name` → `Customer` , `net_revenue` → don't expose it at all (it becomes a measure).
- **Format** measures once, in the model: currency with no decimals for rupees, one decimal for percentages. Then every visual inherits it.
- **Sort** what needs sorting, and set **Data category** for geography columns (City, Region) so map visuals work.

Watch out: don't build a flat table. New users often load one wide table with customer, product, and date columns repeated on every row, because it looks like a spreadsheet. It works at 200,000 rows and collapses later: filters get slow, totals get wrong when the same customer appears in two rows, and time intelligence has nothing to hang on. Build the star.

16.4 DAX from zero

DAX (Data Analysis Expressions) is the formula language of Power BI, Power Pivot (Chapter 11, section 11.8), and Analysis Services. It looks like Excel formulas and behaves differently in one crucial way: **a DAX formula has no cell**. It runs inside whatever filters the visual applies.

Measures and calculated columns

	Calculated column	Measure
Computed	Once per row, at refresh	Once per visual cell, at view time
Stored	In the model (costs memory)	Not stored
Sees	The current row	The current filter context
Use for	Something you slice or group by (a band, a flag, a sorted key)	Anything you aggregate (revenue, margin, counts, ratios)

The rule of thumb: **if you'd put it on an axis or in a slicer, it's a column; if you'd put it in the values area, it's a measure**. Beginners write columns for everything, which bloats the model and gives wrong totals (a column of "margin %" per row, averaged, is not the margin).

Riverstone's base measures, with their 2025 values:

```

Net Revenue =
SUMX ( Sales, Sales[quantity] * Sales[unit_price] * ( 1 - Sales[discount_pct] / 100 ) )

Product Cost = SUMX ( Sales, Sales[quantity] * RELATED ( Product[unit_cost] ) )

Gross Margin = [Net Revenue] - [Product Cost]

Gross Margin % = DIVIDE ( [Gross Margin], [Net Revenue] )

Orders = DISTINCTCOUNT ( Sales[order_id] )

Customers = DISTINCTCOUNT ( Sales[customer_id] )

Average Order Value = DIVIDE ( [Net Revenue], [Orders] )

```

Four things to notice:

- **SUMX is an iterator:** it walks the table row by row, computes the expression, and sums the results. Use it when the calculation happens per row before aggregating. **SUM** alone can only add an existing column.
- **RELATED** reaches from the fact table to the "one" side of a relationship: each Sales row can see its Product row.
- **DIVIDE** returns blank (or a value you choose) instead of an error when the denominator is zero. Always use it instead of `/`.
- **Measures reference measures.** **Gross Margin %** doesn't repeat the revenue formula; it calls it. When the definition of revenue changes, it changes in one place.

Check your measures against SQL

A measure you can't verify is a guess. Every number in this chapter has a SQL equivalent you can run on `riverstone_full`:

```

SELECT ROUND(SUM(net_revenue)) AS all_years, ROUND(SUM(net_revenue) FILTER (WHERE order_date >=
'2025-01-01')) AS y2025,
    COUNT(DISTINCT order_id) FILTER (WHERE order_date >= '2025-01-01') AS orders_2025,
    COUNT(DISTINCT customer_id) FILTER (WHERE order_date >= '2025-01-01') AS customers_2025,
    ROUND(SUM(net_revenue) FILTER (WHERE order_date >= '2025-01-01') / COUNT(DISTINCT order_id)
FILTER (WHERE order_date >= '2025-01-01'), 2) AS aov_2025,
    ROUND(100*(1 - SUM(product_cost) FILTER (WHERE order_date >= '2025-01-01') /
SUM(net_revenue) FILTER (WHERE order_date >= '2025-01-01')), 1) AS gm_2025
FROM sales_lines;

```

all_years	y2025	orders_2025	customers_2025	aov_2025	gm_2025
2663490235	1146641651	46356	4599	24735.56	27.5

(1 row)

So in a card filtered to 2025, your measures must read: Net Revenue **₹1,146,641,651**, Orders **46,356**, Customers **4,599**, Average Order Value **₹24,735.56**, Gross Margin % **27.5%**. With no year filter, Net Revenue is **₹2,663,490,235** for 2023–2025. [companion/ch16/expected_measures.md](#) lists these and the rest; build the card, compare, and only then move on.

Watch out: COUNTRROWS(Sales) is not orders. The fact table is order **lines**: 209,006 rows for 46,356 orders in 2025. Counting the wrong grain is the most common wrong number in a new model, and it looks plausible.

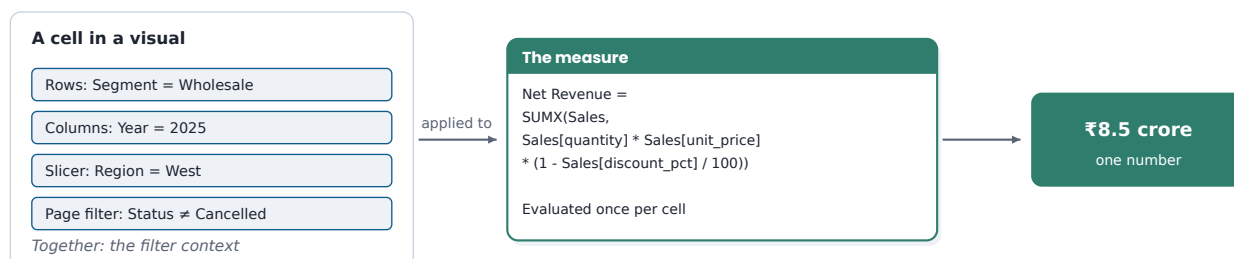
Writing DAX that people can read

- **One measure per idea**, named in business language (`Net Revenue` , not `Sum of amt2`).
- **Format the code**: DAX formatters (the `SHIFT+ENTER` line breaks in the formula bar, or the free DAX Formatter website) turn a wall of text into readable blocks. Team conventions matter more than which convention.
- **Keep measures in a dedicated table**. Create an empty table (**Home** → **Enter data**, call it `_Measures`), move measures into it, and hide its dummy column: all measures then sit in one place in the field list.
- **Write a description** for each measure (in the model view) so the field list explains itself.

16.5 CALCULATE and filter context

Everything hard about DAX is filter context. Everything good about DAX is that you can change it on purpose.

Where a measure's filters come from



CALCULATE changes the filter context before the measure runs

`CALCULATE([Net Revenue], Customer[segment] = "Retail")`
replaces the Segment filter · `REMOVEFILTERS()` clears it · `KEEPFILTERS()` adds to it instead of replacing
`CALCULATE([Net Revenue], DATESYTD(Date[date]))` replaces the date filter with "year so far"

Figure 16.3 – A measure is evaluated once per cell, under the filters that cell carries. `CALCULATE` is how you change those filters.

The idea

A visual cell says "Wholesale, 2025, West region". Those filters reach the model, narrow the tables, and then the measure runs over what's left. Change the cell, and the same measure runs again over a different subset. That's why `[Net Revenue]` in a card is company revenue, and the same measure in a matrix row is that row's revenue: one definition, many answers.

CALCULATE

`CALCULATE ( <expression>, <filter1>, <filter2>, ... )` evaluates the expression with the filters modified:

```
Retail Revenue = CALCULATE ( [Net Revenue], Customer[segment] = "Retail" )

Revenue excl. Cancelled = CALCULATE ( [Net Revenue], Sales[status] <> "Cancelled" )

Company Revenue = CALCULATE ( [Net Revenue], REMOVEFILTERS ( Customer ) )

Share of Segment = DIVIDE ( [Net Revenue], CALCULATE ( [Net Revenue], REMOVEFILTERS ( Customer[customer_name] ) ) )
```

Three rules that explain most surprises:

1. **A filter argument replaces the existing filter on that column.** In a matrix row for Wholesale, `Retail Revenue` shows Retail's number, not blank, because the segment filter was replaced. Wrap the filter in `KEEPFILTERS()` to add to the existing filter instead.
2. **REMOVEFILTERS (or ALL) clears filters**, which is how you compute a denominator for a share. `ALLSELECTED` clears only the filters inside the visual, keeping slicers: use it when a percentage should respond to what the user selected.
3. **CALCULATE also turns a row context into a filter context** ("context transition"), which is why a measure used inside an iterator such as `SUMX` behaves as if each row were filtered.

Filter functions worth knowing early

Function	Does
<code>FILTER (table, condition)</code>	Returns a filtered table; use inside <code>CALCULATE</code> for conditions on measures (<code>FILTER (Customer, [Net Revenue] > 1000000)</code>)
<code>REMOVEFILTERS / ALL</code>	Clears filters from a table or columns
<code>ALLSELECTED</code>	Clears visual filters but respects slicers
<code>KEEPFILTERS</code>	Adds a filter instead of replacing
<code>VALUES / SELECTEDVALUE</code>	The values currently in context; <code>SELECTEDVALUE</code> returns a single one or a default
<code>HASONEVALUE</code>	Tests whether exactly one value is in context (useful for showing a message instead of a wrong total)
<code>RANKX</code>	Ranks; needs a table argument, usually <code>ALL (Customer)</code>

A worked example: the share each product takes of a filtered selection.

```
Product Share % =
VAR ThisProduct = [Net Revenue]
VAR AllProducts = CALCULATE ( [Net Revenue], REMOVEFILTERS ( Product ) )
RETURN DIVIDE ( ThisProduct, AllProducts )
```

In 2025, that gives Storage Box 25L **20.2%**, Food Container Set **18.7%**, Storage Box 10L **17.1%**, and Industrial Crate **13.3%**, which the SQL in section 16.8 confirms.

VAR ... RETURN stores a value once, makes the code readable, and avoids recomputing. Use variables freely; they're evaluated where they're declared, which is itself a useful property when filter context changes later in the formula.

Watch out: totals aren't the sum of the rows. A measure is evaluated at the total row under the total row's filters, not by adding the visible rows. For ratios that's correct behavior (the total margin % is not the sum of the rows' margin %), and for some patterns it looks wrong (a "rank" total, a per-row **IF** that doesn't apply at the total). When a total looks odd, ask what filters the total row has, and use **HASONEVALUE** to blank it out when the total is meaningless.

16.6 Time intelligence

Business questions are mostly comparisons over time: how are we doing this year so far, against last year, against plan. DAX has functions for this, and they all need the date table from section 16.3.

```
Revenue YTD = TOTALYTD ( [Net Revenue], 'Date'[Date] )

Revenue LY = CALCULATE ( [Net Revenue], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )

Revenue YTD LY = CALCULATE ( [Revenue YTD], SAMEPERIODLASTYEAR ( 'Date'[Date] ) )

YoY Growth % = DIVIDE ( [Net Revenue] - [Revenue LY], [Revenue LY] )

Revenue 3M Rolling =
CALCULATE ( [Net Revenue], DATESINPERIOD ( 'Date'[Date], MAX ( 'Date'[Date] ), -3, MONTH ) )

Target = SUM ( Targets[target_revenue] )

% of Target = DIVIDE ( [Net Revenue], [Target] )

Gap to Target = [Net Revenue] - [Target]
```

Check them against SQL. Year to date at the end of June 2025, and the same period last year:

```
SELECT ROUND(SUM(net_revenue) FILTER (WHERE order_date BETWEEN '2025-01-01' AND '2025-06-30')) AS
ytd_jun_2025,
       ROUND(SUM(net_revenue) FILTER (WHERE order_date BETWEEN '2024-01-01' AND '2024-06-30')) AS
ytd_jun_2024,
       ROUND(100.0*SUM(net_revenue) FILTER (WHERE order_date BETWEEN '2025-01-01' AND '2025-06-
30')/SUM(net_revenue) FILTER (WHERE order_date BETWEEN '2024-01-01' AND '2024-06-30') - 100, 1) AS
yoy_pct
FROM sales_lines;
```

```
ytd_jun_2025 | ytd_jun_2024 | yoy_pct
-----+-----+-----
    499203925 |    379844027 |    31.4
(1 row)
```

And by quarter, so you can check a matrix of **Net Revenue** , **Revenue LY** , and **YoY Growth %** :

```
SELECT 'Q' || EXTRACT(QUARTER FROM order_date) AS quarter,
       ROUND(SUM(net_revenue) FILTER (WHERE EXTRACT(YEAR FROM order_date)=2025)) AS rev_2025,
       ROUND(SUM(net_revenue) FILTER (WHERE EXTRACT(YEAR FROM order_date)=2024)) AS rev_2024,
       ROUND(100.0*SUM(net_revenue) FILTER (WHERE EXTRACT(YEAR FROM
order_date)=2025)/SUM(net_revenue) FILTER (WHERE EXTRACT(YEAR FROM order_date)=2024)-100,1) AS
yoy_pct
FROM sales_lines WHERE order_date >= '2024-01-01' GROUP BY 1 ORDER BY 1;
```

quarter	rev_2025	rev_2024	yoy_pct
Q1	264787166	200889263	31.8
Q2	234416760	178954764	31.0
Q3	223564918	175230767	27.6
Q4	423872808	347940682	21.8

(4 rows)

Growth slowed through the year, from 31.8% in Q1 to 21.8% in Q4, which is the honest version of the story Chapter 15's management pack got wrong.

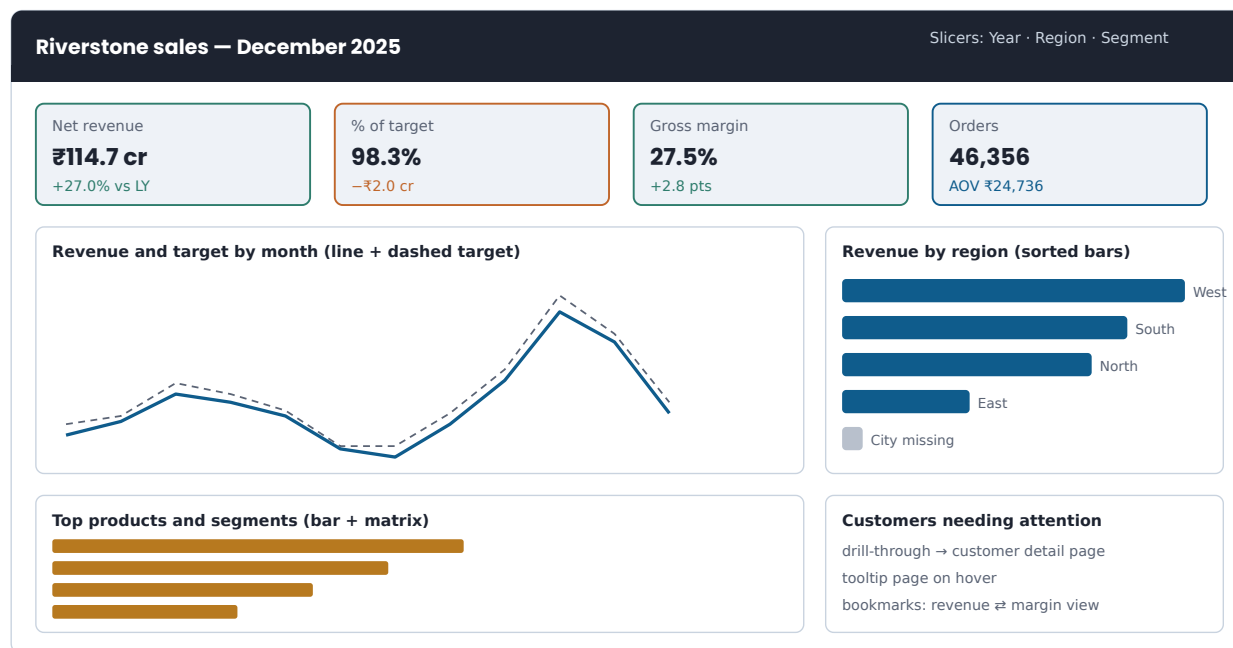
Notes that save hours:

- **Time intelligence needs a marked date table with complete years.** If `Revenue LY` is blank everywhere, that's the first thing to check.
- **SAMEPERIODLASTYEAR** shifts whatever dates are in context, so it works for a month, a quarter, or a custom selection. `DATEADD` is the flexible version (`DATEADD ( 'Date'[Date], -1, YEAR )`).
- **For a 4-4-5 or financial-year calendar**, the built-in functions won't match the business calendar; build the comparison with your own date columns (`Financial Year`, `Period`) and `CALCULATE`.
- **Blank isn't zero.** `Revenue LY` for January 2023 is blank because there's no 2022 data, and a growth % against blank is meaningless. Wrap it: `IF ( NOT ISBLANK ( [Revenue LY] ), ... )`.

16.7 Designing the report page

Chapter 15's principles apply unchanged: start from the question, sort bars, label directly, use one highlight color, write titles that state findings, and never truncate a bar axis. What's new is that the reader can now **interact**, so the design has to guide that too.

The monthly pack as one report page



Four cards answer "how are we doing?" in one line; the visuals below answer "why?". Everything else lives on drill-through pages.

Figure 16.4 — Riverstone's monthly pack as one page: four cards answer "how are we doing?", the visuals below answer "why?", and everything else lives on drill-through pages.

Layout

- **Follow the reading order:** most important at the top left, supporting detail below and to the right. A reader who stops after three seconds should still have the headline.
- **Use a grid.** Align visuals to the same edges and give them the same padding (**Format → General → Properties** for exact position and size). Misalignment reads as carelessness.
- **One page, one question.** "How did we do this month?" is a page. "Which customers are slipping?" is another page, reached by drill-through.
- **Keep cards to four or five.** Each card needs a comparison, or it's a number without meaning: revenue with % of target, margin with change in points.
- **Leave white space.** A page with twelve visuals is a wall; a page with six is a report.

The visuals

Power BI's standard visuals map directly onto Chapter 15's chart types: **column/bar**, **line**, **combo**, **scatter**, **map**, **matrix** (a pivot table), **card** and **KPI**, **gauge** (avoid: it uses angle and area to show one number), **pie** and **donut** (rarely), **treemap**, **waterfall**, **funnel**, **ribbon**, **decomposition tree**, and **key influencers**.

Three that earn their place in a sales report:

- **Matrix** for the numbers people want to read exactly: region by month, with conditional formatting as a heatmap (Chapter 15, section 15.8).
- **Waterfall** for "why did revenue change?" (Chapter 15, section 15.7): set the breakdown to Segment or Product.

- **Decomposition tree** for exploring a number down its dimensions, with the **AI split** option that suggests the dimension with the biggest difference. Useful in a meeting; not a substitute for a designed page.

Custom visuals from AppSource fill gaps (small multiples, better bullet charts). Before using one in a shared report, check who maintains it, whether it's certified by Microsoft, and whether it sends data anywhere.

Interaction

- **Slicers** filter a page. Keep two or three (Year, Region, Segment), and set **Edit interactions** so that a slicer or a visual filters the visuals it should. A common design: a chart filters its neighbours (**Filter**), but cards stay on the whole selection (**None**).
- **The Filters pane** holds filters at three levels: visual, page, and all pages. Use "all pages" for the rule that never changes (exclude cancelled orders, if your fact table hasn't already).
- **Drill-down** (the arrows on a visual) moves through a hierarchy: Year → Quarter → Month, or Region → City.
- **Drill-through** sends the reader to a detail page with the clicked context: right-click a region bar → **Drill through** → **Customer detail**, and that page opens filtered to the region.
- **Tooltips** can be a whole mini-page (**Page information** → **Allow use as tooltip**): hovering a bar shows a small trend chart for that bar's category.
- **Bookmarks** save a state (filters, which visuals are visible) and can be attached to buttons: one button for "revenue view", another for "margin view", on the same page.
- **Buttons, selection pane, and the sync-slicers pane** are how a page becomes a small application. Use them sparingly; every hidden state is something a reader can get stuck in.

Themes and formatting

Set the **View** → **Themes** → **Customize current theme** once: colors (Chapter 15's palette, checked for color vision deficiency), fonts, and default title formatting. Export it as a JSON theme file and reuse it in every Riverstone report so charts look the same everywhere. Individual visual formatting overrides the theme, which is exactly how inconsistency creeps in; prefer changing the theme.

Accessibility, mobile, and performance of the page

- **Alt text** on each visual (**Format** → **General** → **Alt text**), stating what it shows and its finding, as in Chapter 15.
- **Tab order** (**Selection pane** → **Tab order**) so keyboard and screen-reader users move through the page sensibly.
- **Mobile layout** (**View** → **Mobile layout**) rearranges visuals into a single column for phones: branch managers will open it on a phone.
- **Fewer visuals is faster**. Each visual is at least one query; a page with twenty visuals and five slicers can take seconds to render.

16.8 A worked example: Riverstone's monthly pack

Here's the whole build, in the order you'd actually do it. Every number below is checkable with SQL, and `companion/ch16/expected_measures.md` has the full list.

- 1. Load.** Connect to `riverstone_full`, select the six tables, **Transform Data**. In Power Query: remove columns you won't use, add `Region` to `Customer` from the city mapping, and (if you didn't use the `sales_lines` view) filter `Sales` to `status <> "Cancelled"`, or keep `status` and let a measure handle it.
- 2. Model.** Build the star from section 16.3, add the DAX date table, mark it, create relationships, hide keys, rename fields.
- 3. Measures.** The base set from section 16.4, then the time intelligence from section 16.6, in a `_Measures` table.
- 4. Page 1, "Overview".** Four cards (Net Revenue, % of Target, Gross Margin %, Orders), a line chart of Net Revenue and Target by month, a sorted bar of revenue by region, a bar of revenue by product, and slicers for Year, Region, and Segment.

With the Year slicer on 2025, the cards must read:

Card	Value
Net Revenue	₹1,146,641,651
% of Target	98.3%
Gross Margin %	27.5%
Orders	46,356
Average Order Value	₹24,735.56
Customers	4,599

5. Check the region bar against SQL:

```
SELECT r.region, ROUND(SUM(s.net_revenue)) AS rev_2025, COUNT(DISTINCT s.customer_id) AS customers
FROM sales_lines s JOIN customers c ON c.customer_id=s.customer_id
LEFT JOIN (VALUES
  ('Mumbai','West'),('Pune','West'),('Ahmedabad','West'),('Surat','West'),('Nashik','West'),
  ('Nagpur','West'),('Indore','West'),('Goa','West'),('Vadodara','West'),('Rajkot','West'),
  ('Thane','West'),('Aurangabad','West'),
  ('Bengaluru','South'),('Chennai','South'),('Hyderabad','South'),('Kochi','South'),
  ('Coimbatore','South'),('Mysuru','South'),('Visakhapatnam','South'),('Madurai','South'),
  ('Mangaluru','South'),('Thiruvananthapuram','South'),
  ('Delhi','North'),('Jaipur','North'),('Lucknow','North'),('Chandigarh','North'),
  ('Udaipur','North'),('Kanpur','North'),('Ludhiana','North'),('Dehradun','North'),('Agra','North'),
  ('Noida','North'),('Gurugram','North'),
  ('Kolkata','East'),('Bhubaneswar','East'),('Patna','East'),('Guwahati','East'),('Ranchi','East'),
  ('Raipur','East')) AS r(city,region) ON r.city=c.city
WHERE s.order_date >= '2025-01-01' GROUP BY 1 ORDER BY 2 DESC;
```

region	rev_2025	customers
West	383840549	1506
South	318624642	1289
North	279250231	1127
East	142256284	586
	22669946	91

(5 rows)

The blank row is the ₹22,669,946 from 91 customers with no city (Chapter 14). In Power BI it appears as a blank category, and Chapter 15's rule applies: rename it (a Power Query step: replace null with "City missing") and show it in gray rather than dropping it.

6. Check the product bar:

```
SELECT p.product_name, ROUND(SUM(s.net_revenue)) AS rev_2025,
ROUND(100*SUM(s.net_revenue)/SUM(SUM(s.net_revenue)) OVER (),1) AS share
FROM sales_lines s JOIN products p ON p.product_id=s.product_id WHERE s.order_date >= '2025-01-01'
GROUP BY 1 ORDER BY 2 DESC LIMIT 4;
```

product_name	rev_2025	share
Storage Box 25L	231104138	20.2
Food Container Set	214380655	18.7
Storage Box 10L	196381989	17.1
Industrial Crate	152683090	13.3

(4 rows)

7. **Page 2, "Customer detail"**, set as a drill-through target on Region: a matrix of customers with Net Revenue, Orders, YoY Growth %, and last order date; a line of that customer's monthly revenue; and the drill-through filter card showing which region the reader came from.

8. **Titles.** Every visual gets a title that states the finding where the finding is stable ("October is the peak in every region"), or a plain descriptive title where the reader's slicers change the story. Dynamic titles use a measure:

```
Overview Title =
"Riverstone sales – " & SELECTEDVALUE ( 'Date'[Year], "all years" ) &
" – " & FORMAT ( [Net Revenue], "₹#,##0,, .0 cr" ) & " at " & FORMAT ( [% of Target], "0.0%" ) & "
of target"
```

Set the visual's title to that measure with **Format → Title → fx → Field value**.

9. **Save, publish, refresh, secure** (sections 16.9 and 16.10).

16.9 Publishing, refresh, and sharing

From a .pbix file to a report the branches trust

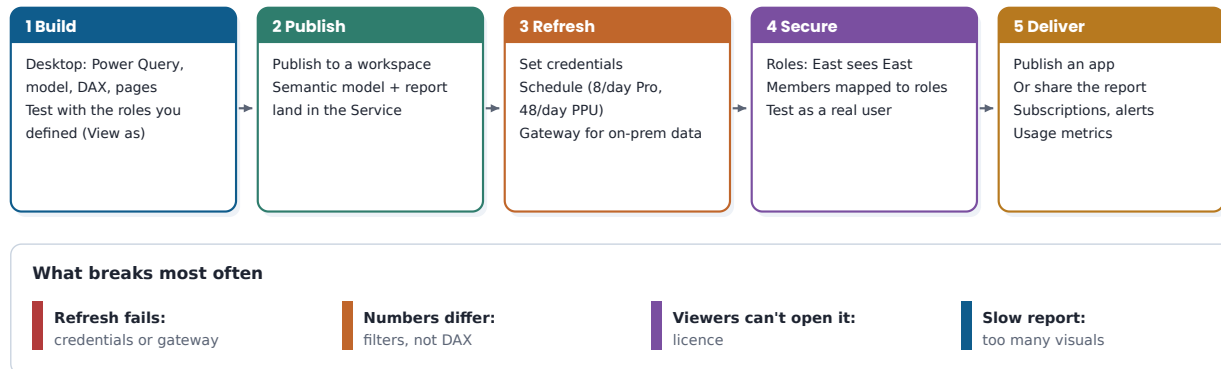


Figure 16.5 – What happens after the report looks right, and what usually goes wrong.

Publish. Home → Publish, choose a workspace. A workspace is a folder with permissions; use one per team or subject area, never "My workspace" for anything shared. Publishing uploads the semantic model **and** the report, and replaces what's there.

Credentials and refresh. In the Service, open the semantic model's settings:

- **Data source credentials:** the account the Service uses to read the source. Personal credentials expire when people leave; ask for a service account.
- **Scheduled refresh:** up to **8 refreshes a day** on Pro, **48** on PPU or Fabric capacity. Set the time to after the source system's nightly load, not at 9:00 when everyone opens the report.
- **Failure notifications:** send them to a team address, not one person.
- **Incremental refresh** (Pro included, larger models on PPU/Fabric): refresh only recent partitions instead of the whole table. Riverstone's 209,006 rows don't need it; a 200-million-row fact table does.

Gateway. A database on the company network is invisible to the cloud. The **on-premises data gateway** (standard mode, installed on a server that stays on) bridges it. Personal mode exists for one user's own refreshes and shouldn't run a company report.

Sharing. Options, from most to least manageable:

1. **An app:** publish the workspace as an app with selected reports and an audience. Readers get a clean, navigable package. This is the default for anything with more than a few readers.
2. **Workspace roles** (Admin, Member, Contributor, Viewer) for the people who build and review.
3. **Share a report link** with specific people, for small, temporary cases.
4. **Embed** in Teams or SharePoint for where people already work.
5. **Subscriptions:** email a PDF or image on a schedule, for readers who won't open a link.

Everyone who opens the content needs a licence: Pro (or PPU, or a free licence if the workspace sits on F64 or larger capacity). Sort this out before you promise a report to 200 people.

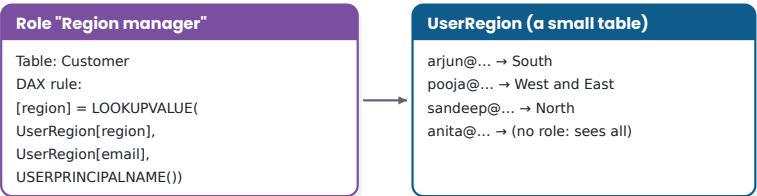
Version control and deployment. Save the `.pbix` in OneDrive, SharePoint, or Git (Chapter 26): Power BI Desktop files are binary, so Git stores them without useful diffs, while the newer `.pbip project format` saves the model and report as text files that diff properly. PPU and Fabric add **deployment pipelines** for development → test → production. At minimum, keep a copy of the last published file and a changelog.

Usage metrics (workspace → report → Usage metrics) show who opens the report and how often. A report nobody opens is a signal, not a failure: ask them what they needed instead.

16.10 Row-level security

Row-level security (RLS) filters the data by who is looking. One report, one model, and the North manager sees North.

Row-level security: one report, four views



What each person sees when they open the same report (2025 net revenue):

Anita Rao (no role)	₹114.7 cr	all 4,599 customers
Pooja Desai (West + East)	₹52.6 cr	West ₹38.4 cr + East ₹14.2 cr
Arjun Nair (South)	₹31.9 cr	1,289 customers
Sandeep Gill (North)	₹27.9 cr	1,127 customers

₹2.3 crore of 2025 revenue belongs to customers with no city, so it has no region: it appears only in the unfiltered view.
Rules filter the dimension; the filter flows to the facts. Security lives in the semantic model, not in the report pages.

Figure 16.6 — Security belongs in the model. The rule filters a dimension, and the filter flows to the facts.

Static roles

In Desktop: **Modeling → Manage roles → Create**, name the role, choose the table, and write a DAX filter:

```
Role "South":    [region] = "South"           on the Customer table
```

Then **Modeling → View as** to test. In the Service, open the semantic model → **Security**, and add people or groups to each role. A user with no role assigned sees everything **if** they have workspace edit rights, and nothing if they don't, which is why testing matters.

Static roles don't scale: 4 regions is fine, 40 sales reps is not.

Dynamic roles

Build a small mapping table (companion/ch16/user_region.csv) with one row per person and region, load it into the model, and write one role:

```
Role "Region manager":  
[region] = LOOKUPVALUE (  
    UserRegion[region],  
    UserRegion[email], USERPRINCIPALNAME (),  
    UserRegion[region], [region]  
)
```

Or, more flexibly, filter Customer with a table expression so one person can own several regions:

```
Role "Region manager":  
Customer[region] IN  
    CALCULATETABLE (  
        VALUES ( UserRegion[region] ),  
        UserRegion[email] = USERPRINCIPALNAME ()  
    )
```

USERPRINCIPALNAME() returns the signed-in user's email in the Service (in Desktop it returns your local account, so test with **View as → Other user**). Add a row to the mapping table and the new manager is covered; no model change, no republish.

Expected results on Riverstone's 2025 data, which is how you test it:

Who	Regions	Net Revenue 2025	Customers
Anita Rao (Sales Head, no role)	all	₹1,146,641,651	4,599
Pooja Desai (RSM West, also covers East)	West, East	₹526,096,833	2,092
Arjun Nair (RSM South)	South	₹318,624,642	1,289
Sandeep Gill (RSM North)	North	₹279,250,231	1,127

The four regions add to ₹1,123,971,706, about ₹22.7 million short of the company total: the customers with no city, and so no region. They appear only in the unfiltered view. Decide deliberately who should see them, and say so on the page.

Rules that keep RLS honest

- **Filter dimensions, not facts**, and let the relationship do the work. A rule on a 209,006-row fact table is slower and easier to get wrong.
- **Test every role**, in Desktop with **View as** and in the Service with **Test as role**, and check a total you know.
- **RLS hides rows, not columns**. Object-level security (column and table level) is separate, and is set outside Desktop's main UI (Tabular Editor).
- **RLS doesn't apply to workspace admins, members, and contributors**: people who can edit the model can see all of it. Viewers, and app users, get the filtered view.
- **Exported data obeys RLS**, and so do subscriptions, which are generated per user.

- **Never build "security" by making a page per region** with visual filters. Anyone can remove a filter, and every new region means a new page.

16.11 Performance and the mistakes that make numbers wrong

Keep the model small and simple

- **Import only what you need** (section 16.2): fewer columns, fewer rows, fewer distinct values. High-cardinality columns (timestamps to the second, GUIDs, free text) cost the most memory.
- **Star schema, not snowflake.** One level of dimension is faster and simpler than dimensions joined to other dimensions.
- **Prefer measures to calculated columns**, and calculated columns to Power Query columns only when they must respond to the model; do heavy transformation upstream in SQL.
- **Avoid bidirectional relationships** unless a specific pattern needs them; they can create ambiguity and slow filtering.
- **Turn off Auto date/time** (File → Options → Data Load → Time intelligence), which silently creates a hidden date table per date column. You have a real one.
- **Watch the visuals per page.** Each visual is a query. A page of twenty visuals, several with complex measures, is where "the report is slow" comes from.
- **Measure it: View → Performance analyzer** in Desktop records how long each visual takes and shows the DAX it ran. DAX Studio (free) goes further, with server timings and a model-size analyzer (VertiPaq Analyzer).

The mistakes that produce wrong numbers

Mistake	Symptom	Fix
Counting fact rows as events	"Orders" shows 83,444 for 2025 instead of 46,356	<code>DISTINCTCOUNT</code> on the order key
Averaging a percentage column	A margin that no month has	Compute the ratio with <code>DIVIDE</code> at the level shown
Cancelled orders left in	Revenue too high by ₹49,241,854 in 2025	Filter in the source view or one page-level filter
No date table, or not marked	Time intelligence blank or wrong	A real date table, marked, covering whole years
Relationship on the wrong column	Totals repeat or collapse	Check both sides' grain in Model view
Bidirectional filters everywhere	Numbers change when unrelated slicers move	One-way filters from dimensions
Blank dimension rows	A category called "(Blank)"	Rename in Power Query ("City missing"), never hide silently
Measures in the fact table's context by accident	Totals don't match rows	Understand context transition; test with <code>HASONEVALUE</code>

Mistake	Symptom	Fix
Different definitions in two reports	Two "revenue" numbers in one meeting	One shared semantic model
Stale data presented as current	Decisions on last week's numbers	Show a last-refresh timestamp on every page

The last one is a measure worth adding to every report:

```
Last Refreshed = "Data refreshed " & FORMAT ( MAX ( 'Date'[Date] ), "d MMM yyyy" ) & " · report refreshed " & FORMAT ( NOW () + TIME ( 5, 30, 0 ), "d MMM yyyy HH:mm" ) & " IST"
```

(NOW() in the Service returns UTC, so add 5 hours 30 minutes for IST, exactly as in Chapter 14, section 14.7.)

16.12 The same ideas in other tools

Every BI tool asks the same four things: connect, model, define measures, design. What changes is where the work happens.

	Power BI	Tableau	Looker Studio	Metabase / Superset
Modelling	Star schema in the model, relationships	Data source with relationships or joins	Blends and joins; heavier work belongs upstream	Mostly upstream, in SQL
Measure language	DAX	Calculated fields and LOD expressions ({FIXED ...})	Calculated fields; LookML in Looker (the enterprise product)	SQL, plus simple metrics
Strength	Modelling, DAX, Microsoft integration, price	Exploratory visual analysis, chart craft	Free, fast for Google-based data	Open source, SQL-first, cheap to self-host
Cost	Free Desktop; per-user or capacity to share	Per-user, higher	Free tier, Pro for governance	Free open source, paid cloud
Runs on	Desktop: Windows only	Windows and macOS	Browser	Browser

Two things transfer completely: **the star schema** and **the discipline of one definition per measure**. The syntax is local knowledge; the modelling is the skill. An analyst who can build Riverstone's model in Power BI can rebuild it in Tableau in a week.

Analysts often meet the others in specific situations: Looker Studio when the data is in Google Analytics or BigQuery and the budget is zero; Metabase or Superset when the company runs its own stack; Tableau in organizations that standardized on it years ago; Excel, still, for the last mile (Chapter 11's Data Model is the same engine as Power BI's).

16.13 Where BI tools are heading

Three shifts matter for an analyst's work, and all three are moving fast enough that you should check the current state before relying on them.

1. **Natural-language questions.** Power BI's Q&A visual and **Copilot** answer typed questions ("revenue by region last quarter") and draft measures, DAX, and report pages. They work best on a **well-modelled, well-named** semantic model, which is the same work this chapter teaches. Treat generated DAX as a draft: verify it against SQL before it reaches a report.
2. **The semantic model as the shared definition.** Fabric, dbt's semantic layer (Chapter 31), and similar tools push toward one place where "net revenue" is defined for every tool. That's a governance win, and it makes the modelling skill more valuable, not less.
3. **BI merging with the data platform.** Fabric's OneLake, lakehouses, and direct-lake connections blur the line between "the warehouse" and "the report". Part III covers the engineering side; for an analyst, the practical effect is fewer imports and more querying of shared tables.

What hasn't changed, and probably won't: someone has to decide what a number means, check it, and design a page that tells the truth in ten seconds.

Common mistakes and how to spot them

Mistake	Symptom	Fix
Loading one flat table instead of a star	Slow filters, wrong totals, no time intelligence	Fact and dimension tables joined by keys
No date table	<code>SAMEPERIODLASTYEAR</code> blank; months sort alphabetically	Build one, mark it, sort month by month number
Calculated columns for everything	Big model, wrong ratios at totals	Measures for anything aggregated
<code>COUNTROWS</code> on the fact table for "orders"	83,444 instead of 46,356	<code>DISTINCTCOUNT (Sales[order_id])</code>
<code>/</code> instead of <code>DIVIDE</code>	Errors or infinity in cards	<code>DIVIDE</code> , with a default if needed
Cleaning inside Power BI that belongs upstream	Every report repeats the same fixes	Fix in the source, a view, or the warehouse
Breaking query folding early	Slow refresh, whole tables pulled	Keep foldable steps first; check View Native Query
Bidirectional relationships by default	Numbers change when unrelated slicers move	One-way from dimensions to facts
Auto date/time left on	Hidden date tables bloat the model	Turn it off in Options
Twenty visuals on a page	Slow page, no message	Fewer visuals, drill-through pages
Chart defaults accepted	Unsorted bars, legends, truncated axes	Chapter 15's rules; a saved theme
Blank categories hidden	Totals don't match the bars	Rename to "City missing", show in gray

Mistake	Symptom	Fix
No refresh timestamp	Old data mistaken for today's	A Last Refreshed measure on every page
Personal credentials on the semantic model	Refresh fails when someone leaves	A service account
"My workspace" for shared content	Nobody else can maintain it	A shared workspace, with an app
Security by separate pages or filters	Anyone can remove a filter	Row-level security in the model
Untested roles	A manager sees nothing, or everything	View as, Test as role, check a known total
Publishing without checking numbers	The first question in the meeting is "is this right?"	Reconcile every card against SQL
Promising a report before checking licences	200 readers, no Pro licences	Confirm licensing first

In the real world: the ₹2.3 crore that vanished

Meera published Riverstone's first Power BI report on a Monday in February 2026. The Overview page had four cards, a revenue-and-target line, a region bar, and a product bar; the numbers matched the ERP exactly, and she'd checked each card against SQL.

On Tuesday, Sandeep Gill added the region bars on his phone during a call and got ₹112.4 crore, while the card above them said ₹114.7 crore. He posted a screenshot in the sales group: "Which number is wrong?" By lunchtime three people had decided the report couldn't be trusted, and one had gone back to the old spreadsheet pack.

Both numbers were right. Ninety-one customers have no city, so they have no region; the region visual dropped them, and the card counted everyone. The gap was ₹2.3 crore, exactly the amount Chapter 14 had flagged and Chapter 15 had insisted on showing.

Meera fixed it in four steps:

1. **Power Query:** replaced null city with "City missing" and null region with "Region missing", so the category exists instead of being blank.
2. **The visual:** the region bar now shows five bars, with "Region missing" in gray at the bottom, and a note in the subtitle: "₹2.3 cr from 91 customers with no city on record."
3. **A reconciliation panel** on a hidden "About this report" page, reached by a small question-mark button: source and refresh time, row counts (209,006 lines, 116,194 orders), what's excluded (cancelled orders: 1,956 orders worth ₹4.9 crore in 2025), and the SQL used to check each card.
4. **A Last Refreshed measure** in the header, because the second question after "is this right?" is always "is this current?".

She also sent the list of 91 customers to the CRM owner, since the fix that matters is upstream.

The report survived. Two months later the branch heads were using drill-through to their own customers, the monthly pack had stopped being assembled by hand, and the only spreadsheet left in the process was the one the plant used for stock.

What made the difference:

- She **showed the gap instead of hiding it**, which converted a credibility problem into a data-quality request.
- She **put the checking on the report itself**: sources, exclusions, counts, refresh time.
- She **treated the first disagreement as feedback**, not an attack. Reports earn trust in the first fortnight or never.

Tools

- **Power BI Desktop** (free, Windows only), current monthly release. macOS or Linux: a Windows VM or cloud desktop.
- **Power BI Service** with a Pro licence or trial, for publishing, refresh, apps, and RLS. Without one, everything up to publishing still works.
- **PostgreSQL 16 or MySQL 8.0** with `riverstone_full` (tested on 16.15 and 8.0.46), or the CSV files in `companion/full/`.
- **Optional**: DAX Studio and Tabular Editor (free) for performance analysis and model editing; the DAX Formatter website.
- **Companion files (`companion/ch16/`)**:
 - `measures.dax` : every measure in this chapter, ready to paste.
 - `expected_measures.md` : each measure's expected value on the full dataset, with the SQL that produces it.
 - `ch16_checks.sql` : that SQL as one runnable script (PostgreSQL; a MySQL version is included).
 - `user_region.csv` : the mapping table for dynamic row-level security.
 - `city_region.csv` : city-to-region mapping for the Customer dimension.
 - `date_table.dax` and `report_checklist.md` : the date table code and a page-review checklist.

The project: replace Riverstone's monthly pack with a live report

Goal: one published report that answers the monthly review's questions, refreshes itself, and shows only each manager's region.

Option A: your own data. Build the same thing on a dataset from your work (anonymized), with at least one fact table, three dimensions, and a date table.

Option B: Riverstone. Use `riverstone_full`.

Steps

1. **Model:** star schema, date table marked, keys hidden, fields renamed, formats set.
2. **Measures:** the base set and the time intelligence, in a `_Measures` table, each with a description.
3. **Check every measure against SQL** using `ch16_checks.sql`. Don't design anything until the numbers match.
4. **Overview page:** four or five cards, revenue vs target by month, revenue by region (including "Region missing"), revenue by product, and slicers for Year, Region, and Segment. Chapter 15's rules throughout; a theme saved and applied.
5. **Detail page:** customers, reached by drill-through from region, with YoY growth and last order date.
6. **About page:** sources, refresh time, exclusions, row counts, and the definition of each measure in business language.
7. **Publish** to a workspace, set credentials, schedule a refresh, and (if you have the licences) publish an app.
8. **Row-level security:** a dynamic role driven by `user_region.csv`. Test that West + East returns ₹526,096,833 and South ₹318,624,642 for 2025.
9. **Write a half-page note to the business:** what the report answers, what it excludes, how often it refreshes, who to ask when a number looks wrong.

What good looks like: every card matches SQL to the rupee; the region bars plus "Region missing" add to the card total; the page renders in under three seconds (Performance analyzer); roles tested; a refresh timestamp visible; no visual without a title that says something.

Stretch goals

- Add incremental refresh on the Sales table, partitioned by year, and compare refresh times.
- Add a "what changed" page: a waterfall of 2024 → 2025 revenue by segment (Retail +₹11.3 crore, Wholesale +₹7.4 crore, Hospitality +₹5.7 crore), with a dynamic title.
- Rebuild the Overview page in Looker Studio or Tableau Public and list, in ten lines, what was easier and what was harder.

Timed challenge: the Q4 branch page (45 minutes)

One page, a clock, and answers you can check. Use `riverstone_full`, a Year slicer set to 2025, and a page filtered to Q4 (October–December).

- **Level 1:** Q4 2025 net revenue, orders, and customers.
- **Level 2:** Q4 2025 average order value and gross margin %.
- **Level 3:** Q4 2025 as a percentage of the Q4 target, and the gap in rupees.
- **Level 4:** Q4 2025 growth over Q4 2024.
- **Level 5:** Q4 revenue by region, sorted, with the missing-region bar shown.
- **Level 6:** the top three products in Q4, and the share of the biggest.
- **Level 7:** revenue with no sales rep in 2025, and what share of the year that is.

- **Bonus:** a dynamic page title that reads "Q4 2025: ₹42.4 cr, 94.7% of target, +21.8% on last year" and updates when the slicer changes.

Answers at the end of the chapter.

You've got it when...

- ☐ You can explain what Desktop, the Service, a workspace, a semantic model, and an app each are, and who needs which licence.
- ☐ You choose Import or DirectQuery for a reason, and keep query folding alive where it matters.
- ☐ You build a star schema with a marked date table, hidden keys, business names, and set formats.
- ☐ You know when to write a measure and when to write a calculated column.
- ☐ You can write `Net Revenue`, `Gross Margin %`, `Orders`, `% of Target`, `Revenue LY`, and `YoY Growth %` from memory, and check each against SQL.
- ☐ You can explain filter context and what `CALCULATE`, `REMOVEFILTERS`, `KEEPFILTERS`, and `ALLSELECTED` do to it.
- ☐ You design a page with a message, Chapter 15's chart rules, a saved theme, drill-through, and alt text.
- ☐ You publish, set credentials and a schedule, know when a gateway is needed, and share through an app.
- ☐ You implement and test dynamic row-level security, and know what it doesn't protect.
- ☐ You can name the five modelling mistakes most likely to make a number wrong, and check for them.

Recap

- **Power BI Desktop** (free, Windows) builds; the **Service** refreshes and shares. Pro is \$14 and PPU \$24 per user per month at the time of writing, with Fabric capacity from F64 letting free users view. Check current pricing before promising anything.
- **Import** beats DirectQuery for anything that fits, and Riverstone's 209,006 lines fit with room to spare.
- **Clean upstream where you can**; use Power Query for what's left, and keep query folding.
- **Model as a star**: facts (order lines, targets), dimensions (Date, Customer, Product, Employee), a marked date table with whole years, keys hidden, business names.
- **Measures, not calculated columns**, for anything aggregated. `SUMX` iterates; `RELATED` reaches across a relationship; `DIVIDE` protects against zero; measures call measures.
- **Filter context is the whole of DAX**. `CALCULATE` replaces filters; `KEEPFILTERS` adds; `REMOVEFILTERS` / `ALL` clear; `ALLSELECTED` respects slicers. Totals are computed under the total's filters, not by adding rows.

- **Time intelligence** needs the date table: `TOTALYTD` , `SAMEPERIODELASTYEAR` , `DATESINPERIOD` .
Riverstone 2025: ₹1,146,641,651, 98.3% of target, +27.0% on 2024, with growth slowing from 31.8% in Q1 to 21.8% in Q4.
- **Design the page** with Chapter 15's rules plus interaction: slicers, drill-through, tooltip pages, bookmarks, a theme, alt text, and a mobile layout.
- **Publishing is a process**: workspace, credentials, schedule, gateway, roles, app. Show a refresh timestamp.
- **Row-level security** lives in the model, filters dimensions, and must be tested against known totals.
- **Most wrong numbers are modelling mistakes**, not DAX mistakes: wrong grain, wrong relationship, blank categories, cancelled orders, or two definitions of revenue.

Practice exercises

Build in Power BI Desktop where you can; check every answer against `riverstone_full` with SQL.

Warm-up

1. Which of these needs Power BI rather than a spreadsheet, and why: (a) a one-off pricing analysis; (b) a weekly stock report for six branches; (c) a monthly sales pack read by 40 people; (d) reconciling one export against an ERP?
2. Name the piece of Power BI that does each job: transforms source data; stores the tables and DAX; holds report pages for a team; lets the cloud refresh from an on-premises database; packages reports for readers.
3. Which licence does each person need: an analyst who only builds on their laptop; an analyst who publishes to a shared workspace; 30 branch managers who only read; 400 readers when the workspace sits on F64 capacity?
4. For each, say measure or calculated column: revenue; order size band (small/medium/large); margin %; financial year; number of distinct customers; days since last order (shown per customer in a table).
5. Why is `COUNTRROWS ( Sales )` the wrong measure for "orders" in Riverstone's model, and what are the two numbers for 2025?
6. What breaks if the date table stops at 15 December 2025?

Core

7. Build the star schema and write the seven base measures from section 16.4. Check all six 2025 values in the table in section 16.8.
8. Add `% of Target` , `Gap to Target` , `Revenue LY` , and `YoY Growth %` . What are the 2025 values of each?
9. Build a matrix of quarter by year with `Net Revenue` and `YoY Growth %` . In which quarter of 2025 was growth slowest, and what was it?
10. Build the region bar chart. What is East's 2025 revenue, and how much revenue has no region?

- l1. Write `Product Share %` with `REMOVEFILTERS` , and check the top four products' shares for 2025.
- l2. Add a segment matrix with `Net Revenue` , `Revenue LY` , and `YoY Growth %` for 2025. Which segment grew fastest?
- l3. Write a measure for revenue from orders with no sales rep, and express it as a share of 2025 revenue.
- l4. What was the value of cancelled orders in 2025, and how many orders? Where in the model should this exclusion live, and why?
- l5. Build the Overview page from section 16.8 and apply a theme with Chapter 15's palette. List three default settings you changed and why.
- l6. Add a drill-through page for customers. Which filters travel with the drill-through, and how do you show the reader what they're looking at?
- l7. Add a `Last Refreshed` measure showing IST, and put it in the page header.
- l8. Create a static role for South and test it with **View as**. What 2025 revenue and customer count do you see?
- l9. Replace it with a dynamic role driven by `user_region.csv` . What does Pooja Desai see, and why does she see two regions?
- l20. Run **Performance analyzer** on your Overview page. Which visual is slowest, and what would you try first to speed it up?

Stretch

- l21. Turn off Auto date/time and compare the model size before and after (**Model view** → **Properties**, or DAX Studio's VertiPaq Analyzer). What changed?
- l22. Write a measure that returns the customer name with the highest revenue in the current filter context, blank when more than one customer is in context. (Hint: `TOPN` , `SELECTEDVALUE` , `HASONEVALUE` .)
- l23. Riverstone's finance team reports on April–March financial years. Add a `Financial Year` column and a `Revenue FYTD` measure that doesn't use `TOTALYTD` . What is FY2026 revenue to 31 December 2025?
- l24. Set up incremental refresh on Sales partitioned by year. What must be true of the source query for it to work?

Think about it

- l25. A manager asks for "the same numbers as the ERP screen, but prettier". What questions do you ask before building?
- l26. Two teams publish two reports whose revenue differs by 4%. How do you find the cause, and what would you change so it doesn't happen again?
- l27. Your report will be read by 300 people who each need a Pro licence. Finance asks whether there's a cheaper way. What options do you present?

Key terms

business intelligence · Power BI Desktop · Power BI Service · workspace · semantic model (dataset) · report · dashboard · app · Microsoft Fabric · Pro · Premium Per User · Fabric capacity (F SKU) · on-premises data gateway · Import mode · DirectQuery · composite model · live connection · query folding · star schema · fact table · dimension table · grain · relationship · cross-filter direction · date table · mark as date table · DAX · measure · calculated column · iterator (SUMX) · RELATED · DIVIDE · filter context · row context · context transition · CALCULATE · REMOVEFILTERS · KEEPFILTERS · ALLSELECTED · VAR / RETURN · time intelligence · TOTALYTD · SAMEPERIODLASTYEAR · DATESINPERIOD · slicer · drill-down · drill-through · tooltip page · bookmark · theme · alt text · mobile layout · scheduled refresh · incremental refresh · deployment pipeline · row-level security · USERPRINCIPALNAME · object-level security · Performance analyzer · DAX Studio · usage metrics · Q&A · Copilot

(All terms are defined in the Glossary, Appendix A.)

Where this leads

- **Chapter 20, Automating Reports & Delivering Insights:** subscriptions, alerts, and scheduled delivery around this report, and how to present what it shows.
 - **Chapter 23, Data Storytelling:** turning a page of visuals into a decision.
 - **Chapter 28, Advanced SQL, Performance & Data Modeling:** star schemas, slowly changing dimensions, and the warehouse the model should eventually read from.
 - **Chapter 31, Analytics Engineering with dbt:** defining measures once, upstream of every BI tool.
 - **Chapter 46:** scheduling and orchestrating the loads that feed a BI model.
 - **Interview preparation:** the Excel, Google Sheets, VBA & BI Question Bank (Chapter 70) covers DAX, filter context, star schemas, and "why don't these two numbers match?".
-

Answers to practice exercises

(In the finished book these move to Appendix G.)

1. (b) and (c) need Power BI: repeated, shared, refreshed. (a) is a one-off for a spreadsheet or notebook. (d) is a cleaning and reconciliation job for SQL or Python (Chapter 14), though its result may feed a report.
2. Power Query transforms; the semantic model stores tables and DAX; a workspace holds team content; the on-premises data gateway bridges to internal sources; an app packages reports for readers.
3. Free (Desktop only); Pro; Pro each (or a capacity); free licences, because F64 and above allow free users to view content in that capacity.

4. Measures: revenue, margin %, distinct customers. Calculated columns: order size band and financial year (you slice by them). "Days since last order" per customer can be either: a column if it's a static attribute refreshed nightly, a measure if it must respond to the date slicer, which is usually what people want.
5. The fact table is order **lines**. In 2025 there are **83,444** lines and **46,356** orders, so `COUNTROWS ( Sales )` overstates orders by about 80%. Use `DISTINCTCOUNT ( Sales[order_id] )`.
6. Year-to-date and last-year comparisons break for December (and anything after the 15th): `TOTALYTD` and `SAMEPERIODLASTYEAR` need complete periods in the date table, and the December column would silently show a partial month.
7. 2025: Net Revenue ₹1,146,641,651 · Orders 46,356 · Customers 4,599 · Average Order Value ₹24,735.56 · Gross Margin % 27.5% · all years ₹2,663,490,235.
8. 2025: % of Target **98.3%** (target ₹1,166,950,000), Gap to Target **-₹20,308,349**, Revenue LY **₹903,015,475**, YoY Growth % **+27.0%**.
9. Q4, at **+21.8%** (Q1 +31.8%, Q2 +31.0%, Q3 +27.6%). Growth slowed all year even though Q4 was the biggest quarter in rupees (₹423,872,808).
10. East **₹142,256,284** (586 customers). Revenue with no region: **₹22,669,946** from 91 customers with no city.
11. Storage Box 25L **20.2%**, Food Container Set **18.7%**, Storage Box 10L **17.1%**, Industrial Crate **13.3%**.
12. Wholesale, **+31.1%** (₹312,728,493 against ₹238,457,929). Retail grew 25.4% and Hospitality 25.7%, so Wholesale gained share.
13. Revenue no rep = `CALCULATE ( [Net Revenue], ISBLANK ( Sales[sales_rep_id] ) )` (or a relationship-aware version with `REMOVEFILTERS ( Employee )` and a blank check). 2025: **₹33,630,135** on 2,468 lines, **2.9%** of the year's revenue. It's why a rep-level page needs an "(unassigned)" row (Chapter 14, section 14.3).
14. **₹49,241,854** across **1,956** orders. The exclusion belongs upstream, in the view or the Power Query filter, so every report shares it; a page-level filter is acceptable in one report, but it will be forgotten in the next one.
15. Any three, with reasons, such as: sorted the region bar by value rather than alphabetically; removed the legend and labeled directly; set the theme's data colors to the Riverstone palette; turned off the automatic axis start on a bar chart; set number formats to crore with one decimal.
16. The filters on the visual you right-clicked plus the page filters travel with the drill-through (Power BI shows them in the Filters pane). Show the reader by putting a text box or card bound to `SELECTEDVALUE ( Customer[region] )` at the top of the detail page, and keep the automatic **Back** button.
17. Last Refreshed = "Report refreshed " & `FORMAT ( NOW () + TIME ( 5, 30, 0 ), "d MMM yyyy HH:mm" )` & " IST". `NOW()` evaluates in UTC in the Service, so the offset is needed; in Desktop it shows your machine's clock.

18. South in 2025: ₹318,624,642 and 1,289 customers. Every visual on every page changes; cards, totals, and the map all shrink.

19. Pooja Desai sees ₹526,096,833 (West ₹383,840,549 + East ₹142,256,284) across 2,092 customers, because `user_region.csv` has two rows for her email: she is the West manager and also covers East until a fourth regional manager is hired.

20. Typically the matrix or the map, because they produce the largest queries. First things to try: reduce the number of visuals, remove unnecessary columns from the visual, replace a complex measure with a simpler one (or a column computed at refresh), and check whether a slicer with many values is the real cost.

21. The model gets smaller, often noticeably, because Power BI stops creating a hidden date table for every date column. On a small model like Riverstone's the saving is a few hundred kilobytes; on a model with twenty date columns and ten years of data it can be tens of megabytes, plus faster refreshes.

22. One solution:

```
Top Customer =
IF (
    HASONEVALUE ( Customer[Customer] ),
    BLANK (),
    SELECTEDVALUE (
        TOPN ( 1, VALUES ( Customer[Customer] ), [Net Revenue], DESC ),
        BLANK ()
    )
)
```

`TOPN` returns a one-row table, `SELECTEDVALUE` turns it into a value, and `HASONEVALUE` blanks it out when a single customer is already in context (where the answer would be trivial). On 2025 data with no other filter, the biggest customer's revenue is about ₹10.7 lakh.

23. Add `Financial Year = "FY" & IF ( MONTH ( [Date] ) >= 4, YEAR ( [Date] ) + 1, YEAR ( [Date] ) )`, then:

```
Revenue FYTD =
CALCULATE (
    [Net Revenue],
    FILTER ( ALL ( 'Date' ), 'Date'[Financial Year] = MAX ( 'Date'[Financial Year] ) &&
        'Date'[Date] <= MAX ( 'Date'[Date] ) )
)
```

FY2026 runs from 1 April 2025, so to 31 December 2025 it is April–December 2025: ₹881,854,486 (verify with `SELECT SUM(net_revenue) FROM sales_lines WHERE order_date BETWEEN '2025-04-01' AND '2025-12-31'`).

24. The source must support query folding so the date filters (`RangeStart` and `RangeEnd` parameters) are pushed to the source, the partitioning column must be a date or datetime, and rows must not change in closed partitions (or you must refresh them too). A database source folds; a folder of CSVs generally doesn't.

25. Ask: which decisions does this support, and who makes them? Which numbers exactly, with what definition and what exclusions? How current must they be (daily, hourly)? Who may see what? Does the ERP screen agree with finance, and which is the definition of record? And what's the one question the page should answer in ten seconds? "Prettier" usually means "I can't find what I need", which is a design question, not a styling one.

26. Compare definitions before comparing charts: the period, the exclusions (cancelled orders, internal accounts), the grain (lines versus orders), the currency and rounding, and the refresh times. Reconcile one small slice (one branch, one month) until you find where the two diverge. Then fix it upstream: one shared semantic model, or one definition in the warehouse (Chapter 31), with the definitions written on an "About" page so the next disagreement takes five minutes.

27. Options: (a) Pro for everyone ($300 \times \$14 \approx \$4,200$ a month at list); (b) Fabric capacity at F64 or above, where viewers can hold free licences, which is often cheaper above roughly 350–600 viewers; (c) reduce the audience: an app for the people who need interaction, and emailed PDF subscriptions or a paginated report for those who only read one page; (d) check what's already owned, since Microsoft 365 E5 includes Pro. Present the break-even calculation, not only the options.

Timed challenge answers. Level 1: ₹423,872,808 · 13,777 orders · 4,220 customers. Level 2: AOV ₹30,766.70 · gross margin 27.6%. Level 3: 94.7% of a ₹447,500,000 target, a gap of –₹23,627,192. Level 4: +21.8% on Q4 2024's ₹347,940,682. Level 5: West ₹142,688,728 · South ₹117,726,527 · North ₹102,966,828 · East ₹52,120,934 · Region missing ₹8,369,792. Level 6: Storage Box 25L ₹89,225,213 (21.0%), Food Container Set ₹83,156,260 (19.6%), Storage Box 10L ₹76,099,616 (18.0%). Level 7: ₹33,630,135, or 2.9% of 2025 revenue. Bonus: build the title from `SELECTEDVALUE` , `[Net Revenue]` , `[% of Target]` , and `[YoY Growth %]` with `FORMAT` , as in section 16.8.